

Ergebnisse des Screenings der Website helinet.de

Autor: Thomas Lonnemann

Datum: 25. Januar 2026

Versionsnummer: 1.0

Inhaltsverzeichnis

Inhaltsverzeichnis 2

Zusammenfassung 3

Welche Module laufen so, nicht unter Contao 5.7 (und warum)? 3

Code-Check - konkrete Probleme (PHP 8.x) und Verbesserungen 4

Was bei “Upgrade/Port nötig” konkret zu tun wäre? 5

KI Analyse - “Denkst Du es ist schneller die Module zu porten oder sauber nur das eigentlich benötigte komplett neu aufzusetzen?” 6

KI-Analyse mit Aufwandsschätzungen der KI 9

Zusammenfassung

Insgesamt lässt sich sagen, dass das aktuelle System ein gewachsenes und an manchen Stellen auch verwachsenes System ist.
Es sind für Kleinigkeiten, wie ein Modal oder Google-Maps-Funktion externe Drittanbieter-Pakete verwendet worden. Ob das Vorgabe oder limitierte Expertise war, können wir nicht sagen und/oder bewerten. Zukünftig muss die Vorgabe aber sein, dass möglichst alle Elemente und/oder Funktionen der Website in eigenem Code erstellt werden sollten, damit laufende Wartung, Dokumentation und Qualität gewährleistet werden können.
Viele Pakete sind vom Stand noch auf **Contao 4.x** festgenagelt (Composer-Requirement contao/core-bundle), d. h. sie **können nicht in Contao 5.7 installiert werden**, solange das System nicht auf neuere Releases wechselt oder diese ersetzt werden.

Insgesamt würden wir das Contao-System mit in etwa 4/10 bewerten:

- **+ Gut/solide (zieht die Note hoch):**
  Ein Teil der Erweiterungen stammt von etablierten Contao-Anbietern (z. B. terminal42, codefog). Die fachliche Idee ist ok, und es gibt für vieles moderne Composer-Bundles.
- **± Mittelmäßig (typisch Legacy):**
  Der Stand ist stark **Contao-3/4-legacy** geprägt (ClassLoader/autoload.php, globale Hooks, wenig DI/Services). Das ist nicht „schlechter Code" im Sinne von völlig kaputt – aber aus heutiger Sicht **wartungsintensiv** und **Upgrade-unfreundlich**.
- **– Schwach (zieht die Note deutlich runter):**
  Mehrere Module sind **abandoned** (z. B. dlh_* / extended_checkbox) und/oder haben offensichtliche **PHP-8-Brüche** (z. B. each() in dlh_googlemaps). Abandoned + PHP-8-Breaks sind ein Qualitäts-Minus.

Aus unserer Sicht ist auf der reinen Code-Grundlage eine komplette Neuinstallation auf einem sauberen und aktuellen System die mit Abstand beste Vorgehensweise.
Vor allem vor dem Hintergrund, dass das Frontend und der Inhalt noch angepasst werden sollen, sind hier Synergien und weniger Aufwand anzunehmen, da nicht alter Code “gerade gebogen” werden muss, sondern die Content-Elemente passgenau gebaut werden können.
Die Server-Architektur haben wir folgendermaßen vorgesehen.
Identische Server für:

1. LIVE
2. STAGE (Testing und Freigabe)
3. DEV (Entwicklung)

Welche Module laufen so, wie sie vorliegen, nicht unter Contao 5.7 (und warum)?

A) Direkt inkompatibel (Composer verlangt Contao 3/4, nicht 5)
Diese Versionen würden in Contao 5.7 **nicht installierbar** sein, weil sie contao/core-bundle nur bis 4.x erlauben:

- dlh_geocode → contao/core-bundle: ^3.5 || ~4.2
- dlh_googlemaps → contao/core-bundle: ^3.5.1 || ~4.2
- euf_overlay → contao/core-bundle: ~3.5 || ~4.4
- extended_checkbox → contao/core-bundle: ~4.4
- haste → contao/core-bundle: ^4.4
- leads → contao/core-bundle: ^3.5.1 || ^4.1
- notification_center → contao/core-bundle: ~4.4
- mp_forms → contao/core-bundle: ^4.9
- catalog-manager → contao/core-bundle: ^4.4

Das ist eine harte Composer-Schranke. Mit diesen composer.json bekommt man sie unter Contao 5.7 nicht aufgelöst.

B) Potentiell kompatibel (Composer erlaubt Contao 5), aber trotzdem nicht "einfach kopieren"

- mobile_menu → contao/core-bundle: ^4.13 || ^5.0 (also Contao 5 grundsätzlich erlaubt)

Warum trotzdem nicht kopieren?
Der Ordner ist noch im **Legacy modules/-Layout** (mit config/autoload.php, ClassLoader etc.).
In Contao 5 ist der saubere Weg: **per Composer installieren** (aktuelles Bundle), nicht diesen Ordner übernehmen.

C) Ohne composer.json (Legacy/custom) → manuelle Portierung nötig

- LC (eigenes helinet-spezifisches Modul)
- pct_autogrid
- jquery-ui-tabs
- m17StickyBEFooter
- zFormDependent
- MandatoryField

In Contao 5 müssen das Bundles/Packages sein (PSR-4 Autoload, src/, Services etc.). Ohne offizielle Bundle-Nachfolger ist das ein sehr aufwändiges Migrationsprojekt.
Benjamin hat bereits angefangen, das LC-Modul zu portieren, das sieht ganz gut aus, ist aber noch nicht fertig und/oder final.

Code-Check - konkrete Probleme (PHP 8.x) und Verbesserungen

Legacy-Patterns, die für Contao 5 modernisiert müssen
Im Code u. a. häufig gefunden:

- **ClassLoader / config/autoload.php** (Legacy-Autoloading)
- **$GLOBALS['TL_HOOKS']… + viele ->import() Aufrufe** (klassische Contao-3/4-Patterns)
- **TL_ROOT-Nutzung** (alt, in modernen Setups nicht mehr möglich)
- **Nicht Contao-5-konforme Ordnerstruktur:**
- Pakete als **Symfony/Contao Bundle** bauen: src/, Namespaces, PSR-4 in Composer
- Hooks möglichst als **EventSubscriber/Services** statt direkter $GLOBALS-Registrierung
- **Dependency Injection** statt ->import()
- Typisierung (mindestens schrittweise): Parameter-/Return-Typen, keine dynamischen Properties, declare(strict_types=1);

Was bei “Upgrade/Port nötig” konkret zu tun wäre?

Wenn ein Modul **nicht** als Contao-5-Bundle verfügbar ist, ist das typische Port-Set:

1. **Bundle/Composer-Grundgerüst**

- composer.json mit PSR-4 Autoload ("autoload": {"psr-4": {"Vendor\\Bundle\\": "src/"}})
- src/…Bundle.php (Symfony Bundle)
- ggf. contao-manager-plugin (Manager-Integration), statt des alten contao-community-alliance/composer-plugin

1. **Dateistruktur an Contao 5 anpassen**

- config/autoload.php entfernen → PSR-4
- dca/… nach contao/dca/…
- languages/… nach contao/languages/…
- templates/… nach contao/templates/…
- Assets (JS/CSS/Images) nach public/… oder über Symfony Asset-Build

1. **Hooks → Services/EventSubscriber**

- Statt $GLOBALS['TL_HOOKS']… in config/config.php:
    - als Services definieren (config/services.yaml)
    - als EventSubscriber/Listener (wo möglich) oder Hook-Service (Contao stellt Brücken bereit)
- Bei pct_autogrid: die Hooks parseTemplate, parseWidget, generatePage etc. sind “sensibel” → hier entstehen die meisten Regressionen.

1. **DCA & Callbacks modernisieren**

- DCA-Callbacks oft als array('Class', 'method') → auf namespaced Klassen + Services umstellen
- Prüfen: onsubmit_callback, onload_callback, save_callback, options_callback etc.

1. **PHP 8.x Fixes**

- entfernte Funktionen (z. B. each() – z.B. in dlh_googlemaps) → refactor
- dynamische Properties vermeiden; Typisierung ergänzen

KI Analyse - “Denkst Du es ist schneller die Module zu porten oder sauber nur das eigentlich benötigte komplett neu aufzusetzen?”

Meist ist **„sauber neu aufsetzen + nur wirklich Benötigtes übernehmen“** schneller – *mit einer wichtigen Ausnahme:* **Custom-/Vendor-Logik**, die es so nirgends mehr gibt (v. a. LC), kann Portierung erzwingen.

Warum neu aufsetzen oft schneller ist:

Das System ist ein **Legacy-modules/-Stand** aus Contao 4.x.
In Contao 5.7 ist der „Happy Path“:

- **Composer/Bundles installieren**
- **Konfiguration + Inhalte migrieren**
- **Abandoned-Module ersetzen**

Wenn man stattdessen „alles portet“, portet man nicht nur Features – man portet vor allem **Alt-Struktur** (autoload.php, globale Hooks, alte DCA-Patterns) → das ist viel Handarbeit und fehleranfällig.

Was bei der Knackpunkt ist:

Grob lassen sich die Modulen in drei Klassen einteilen:

1. **Einfach neu installieren (schnell)**
   notification_center, mp_forms, leads, haste, mobile_menu, jquery-ui-tabs, m17StickyBEFooter
   In Contao 5.7 via Composer neu rein, testen.
   Typisch: 0.5–2 Tage inkl. Tests, je nach Umfang der Formulare/Mailstrecken.
2. **Ersetzen (mittlerer Aufwand, aber klarer Weg)**
   dlh_googlemaps, dlh_geocode (abandoned), extended_checkbox (abandoned), ajaxform (oft obsolet)
   Der Aufwand steckt weniger im Code, mehr in **“wie lösen wir das Feature heute?”**
   Typisch: 0.5–4 Tage, je nachdem wie tief Maps/Geocode in Templates/Content eingebaut ist.
3. **Port nötig (hier steckt die Zeit)**
   LC (custom), pct_autogrid (vendor/unklar)
   Das sind die einzigen, bei denen „neu aufsetzen“ nicht automatisch hilft, weil es ggf. **kein Ersatz** gibt.
   Typisch:

- LC: **1–3 Tage**
- pct_autogrid: **2–6+ Tage** (weil viele Hooks/Templates, Layout-Eingriff)

Die realistische Antwort: Neu aufsetzen ist schneller – und man portet nur 1–2 Module

Wenn du „sauber neu" machst, portierst du am Ende wahrscheinlich nur:

- **LC** (wenn wirklich gebraucht)
- **pct_autogrid** *nur*, falls es kein Contao-5-Bundle vom Hersteller gibt oder ihr exakt diese Logik braucht

Alles andere: **Composer neu installieren oder ersetzen**.

Grobe Zeitrechnung (Faustformel)

A) Alles porten (Worst-Case, legacy → bundle überall):

- Viele Module: je 0.5–2 Tage Port+Tests + Sonderfälle → schnell **2–4 Wochen** Arbeit.

B) Neu aufsetzen + nur benötigte Funktionen übernehmen (typisch):

- Core-Setup + Templates/Theme-Anpassung + Composer-Module: **3–8 Tage**
- Ersatz für abandoned (Maps/Checkbox): **2–8 Tage**
- Port LC (+ ggf. autogrid): **2–10 Tage**
  Summe oft **1.5–3 Wochen**, aber mit **deutlich weniger Risiko** und besserer Wartbarkeit.

Und wenn pct_autogrid ersetzbar ist (Grid/Utilities heute oft anders lösbar), kann's auch deutlich darunter liegen.

Ein einfacher Entscheidungsrahmen:

Neu aufsetzen ist schneller, wenn:

- ≥ 50% der Module per Composer verfügbar sind (gegeben: ja)
- abandoned Module dabei sind (gegeben: ja)
- das System soll „modern" rauskommen (Wartbarkeit/Upgrades)

Porten lohnt sich nur, wenn:

- ein Modul **einzigartige Logik** enthält, die 1:1 gebraucht wird (gegeben: LC, evtl. pct_autogrid)
- die Site extrem stark von genau diesem Modul abhängt und Ersatz teurer wäre als Port

Meine Empfehlung für (praktisch)

1. **Neu aufsetzen (Contao 5.7 / PHP 8.5)**
2. **Nur wirklich benötigte Features inventarisieren** (aus jedem Modul: "Was nutzen wir?")
3. **Composer-Neuinstallation** für alle verfügbaren Bundles
4. **Ersatzentscheidung für Maps/Geocode** (das ist der größte "Produkt"-Entscheidungsblock)
5. **LC porten** (klein, kontrollierbar)

1. pct_autogrid: nur porten, wenn kein einfacher Ersatz möglich ist

KI-Analyse mit Aufwandsschätzungen der KI

Migrations-Matrix (16 Module)

Modul (Ordner)	Empfehlung	Paket / typischer Composer-Require	Wichtigste Port-/Anpassungspunkte im Code	Aufwand (h)
ajaxform	**Ersatz / Weglassen**	**Weglassen** (Funktionalität ist i. d. R. im Core/modernen Setups abgedeckt); falls zwingend: composer require terminal42/contao-ajaxform (aber i. d. R. nicht mehr nötig)	Im derzeitigen Stand legacy config/autoload.php + Templates + DCA – in C5 lieber Core-Mechanismus oder anderes modernes Handling nutzen.	**1–4** (je nach Formularen/JS)
catalog-manager	**Upgrade via Composer möglich** (aber **Daten/Features prüfen**)	Aktueller Stand ist meist als Bundle verfügbar (z. B. alnv/catalog-manager-bundle – je nach Distribution/Branch). Dein Ordner ist alt (alnv/catalog-manager mit C4 Constraint).	Sehr hook-lastig (viele $GLOBALS['TL_HOOKS']), sehr viele Templates (41) + DCA (12). Beim Upgrade: Hook-Verhalten und Template-Overrides prüfen, DB-Migrationen & Backend-Module testen.	**12–30**

dlh_geocode	**Ersatz nötig** (abandoned)	Abandoned – kein offizieller Ersatz vorgeschlagen.	Unter PHP 8.x ok, aber in C5 nicht installierbar (C3/4 constraint). Wenn Geocoding gebraucht: eigene kleine Service-Integration (API) oder neue Map-Extension wählen.	**4–16** (abhängig von Nutzung)
dlh_googlemaps	**Ersatz nötig** (abandoned)	Abandoned; es gibt ein "revived demo" Paket composer require contao-graveyard/googlemaps, wird aber explizit nicht für Produktion empfohlen.	Im Code existiert each() (PHP 8 entfernt) → sofort fatal. Viele Templates (24) + DCA (8). Realistisch: ersetzen (z. B. eigene Maps-Einbindung oder andere Extension).	**8–30**
euf_overlay	**Upgrade via Composer möglich** (aber C5-Tauglichkeit prüfen)	composer require erdmannfreunde/euf_overlay	Legacy-Modul mit DCA+Templates; prüfen ob Paket/Release wirklich Contao-5-fähig ist (sonst Port). Test: Ausgabe/JS/Cookie-Handling.	**3–10**
extended_checkbox	**Ersatz**	Autor empfiehlt plenta/contao-extended-checkbox (statt abandoned Paket).	DCA/Form-Field Anpassungen + ggf. Template/CSS. Mapping: welches Widget/Field wird erweitert? Regressionstest in Formularen.	**2–6**
haste	**Upgrade via Composer möglich**	composer require codefog/contao-haste (aktuelle Version für Contao 5 wählen)	Viele Hooks (u. a. replaceInsertTags, modifyFrontendPage, loadDataContainer). Bei Upgrade prüfen: InsertTags/Undo/Image-Hooks, ggf. Namespaces/DI.	**2–6**

jquery-ui-tabs	**Upgrade via Composer möglich**	composer require wangaz/contao-jquery-ui-tabs	Dein Ordner ist legacy; in C5: Bundle installieren, Templates (8) prüfen/überschreiben falls ihr custom habt.	**1–4**
LC	**Port nötig (Custom)**	—	Kein Composer-Metapaket, legacy config/autoload.php/config/config.php, Hook processFormData. Port: Bundle-Skeleton, PSR-4, Service für Hook/EventSubscriber, evtl. DCA/Model falls vorhanden.	**8–24**
m17StickyBEFooter	**Upgrade via Composer möglich**	composer require jrgregory/m17-sticky-backend-footer	Dein Ordner ist "nackt"/legacy; in C5 über Bundle installieren. Test im Backend (JS/CSS).	**0.5–2**
mobile_menu	**Upgrade via Composer möglich**	composer require codefog/contao-mobile_menu	Hat bereits src/, aber dein Paketstand ist legacy-installiert. Prüfen: Templates/DCA (je 1), Frontend-Rendering + CSS/JS.	**1–4**
mp_forms	**Upgrade via Composer möglich**	composer require terminal42/contao-mp_forms	Hooks compileFormFields, loadFormField, prepareFormData, replaceInsertTags. Beim Upgrade: Formularlogik + MultiPage Flow testen.	**3–8**
notification_center	**Upgrade via Composer möglich**	composer require terminal42/notification_center	Viele Hooks (u. a. processFormData, createNewUser, updatePersonalData). Test: Token/Message-Config, Versandwege, Integrationen mit mp_forms/leads.	**4–12**

leads	**Upgrade via Composer möglich**	composer require terminal42/contao-leads	Hooks rund um Speicherung/Export (processFormData, Export-Row, Backend-Navigation). Prüfen: DB-Schema/Migration, Exporter/Dateitokens.	**4–12**
pct_autogrid	**Port nötig (vendor / kein Bundle im Ordner)**	— (Hersteller/Bundle-Nachfolger prüfen)	Hook-schwer (generatePage, parseTemplate, parseWidget, getContentElement, isVisibleElement…), viele Templates (16) + DCA (4). Ohne offiziellen Contao-5-Release: Portierung als Bundle + eventuelle JS/CSS-Assets nach public/.	**16–50**
zFormDependentMandatoryField	**Ersatz via Composer**	composer require cliffparnitzky/form-dependent-mandatory-field	Dein Ordner ist legacy mit Hooks loadFormField/validateFormField. In C5: Bundle installieren, testen: Pflichtfeldlogik, MulticolumnWizard-Abhängigkeit (steht im Paket).	**2–6**